

Part 9. Type Modifiers

By now you've read all about how the TypeScript type system works with existing JavaScript constructs such as arrays, classes, and objects. For this part and Part 10 - Generics, I'm going to take a step further into the type system itself and show features that focus on writing more precise types, as well as types based on other types.

Top Types

I mentioned the concept of a *bottom type* back in Part4 - Objects to describe a type that can have no possible values and can't be reached. It stands to reason that the opposite might also exist in type theory. It does!

A *top type*, or universal type, is a type that can represent any possible value in a system. Values of all other types can be provided to a location whose type is a top type. In other words, all types are assignable to a top type.

any, Again

The `any` type can act as a top type, in that any type can be provided to a location of type `any`. `any` is generally used when a location is allowed to accept data of any type, such as the parameters to `console.log`:

```
let anyValue: any;
anyValue = "Lucille Ball"; // Ok
anyValue = 123; // Ok

console.log(anyValue); // Ok
```

The problem with `any` is that it explicitly tells TypeScript not to perform type checking on that value's assignability or members. That lack of safety is useful if you'd like to quickly bypass TypeScript's type checker, but the disabling of type checking reduces TypeScript's usefulness for that value.

For example, the `name.toUpperCase()` call below definitely will crash, but because `name` is declared as `any`, TypeScript does not report a type complaint:

```
function greetComedian(name: any) {
    // No type error...
    console.log(`Announcing ${name.toUpperCase()}!`);
}

greetComedian({ name: "Bea Arthur" });
// Runtime error: name.toUpperCase is not a function
```

If you want to indicate that a value can be anything, the `unknown` type is much safer.

unknown

The `unknown` type in TypeScript is its true top type. `unknown` is similar to `any` in that all objects may be passed to locations of type `unknown`. The key difference with `unknown` is that TypeScript is much more restrictive about values of type `unknown`:

- TypeScript does not allow directly accessing properties of `unknown` typed values.
- `unknown` is not assignable to types that are not a top type (`any` or `unknown`).

Attempting to access a property of an `unknown` typed value, as in the following snippet, will cause TypeScript to report a type error:

```
function greetComedian(name: unknown) {
    console.log(`Announcing ${name.toUpperCase()}!`);
    //
    // Error: Object is of type 'unknown'.
}
```

The only way TypeScript will allow code to access members on a name of type `unknown` is if the value's type is narrowed, such as using `instanceof` or `typeof`, or with a type assertion.

This code snippet uses `typeof` to narrow `name` from `unknown` to `string`:

```
function greetComedianSafety(name: unknown) {
    if (typeof name === "string") {
        console.log(`Announcing ${name.toUpperCase()}!`); // Ok
    } else {
        console.log("Well, I'm off.");
    }
}
```

```

    }
}

greetComedianSafety("Betty White"); // Logs: Announcing BETTY WHITE!
greetComedianSafety({}); // Logs: Well, I'm off.

```

Those two restrictions make `unknown` a much safer type to use than `any`. You should generally prefer using `unknown` instead of `any` when possible.

Type Predicates

I've previously shown you how JavaScript constructs such as `instanceof` and `typeof` can be used to narrow types. That's all fine and good for directly using that limited set of checks, but it gets lost if you wrap the logic with a function.

For example, this `isNumberOrString` function takes in a value and returns a boolean indicating whether the value is a `number` or `string`. We as humans can infer that the `value` inside the `if` statement must therefore be one of those two types since `isNumberOrString(value)` returned true, but TypeScript does not. All it knows is that `isNumberOrString` returns a boolean—not that it's meant to narrow the type of an argument:

```

function isNumberOrString(value: unknown) {
    return ['number', 'string'].includes(typeof value);
}

function logValueIfExists(value: number | string | null | undefined) {
    if (isNumberOrString(value)) {
        // Type of value: number | string | null | undefined
        value.toString();
        // Error: Object is possibly undefined.
    } else {
        console.log("Value does not exist:", value);
    }
}

```

TypeScript has a special syntax for functions that return a boolean meant to indicate whether an argument is a particular type. This is referred to as a *type predicate*, also sometimes called a “user-defined type guard”: you the developer are creating your own type guard akin to `instanceof` or `typeof`. Type predicates are

commonly used to indicate whether an argument passed in as a parameter is a more specific type than the parameter's.

Type predicate's return types can be declared as the name of a parameter, the `is` keyword, and some type:

```
function typePredicate(input: WideType): input is NarrowType;
```

We can change the previous example's helper function to have an explicit return type that explicitly states `value is number | string`. TypeScript will then be able to infer that blocks of code only reachable if `value is number | string` is `true` must have a `value` of type `number | string`. Additionally, blocks of code only reachable if `value is number | string` is `false` must have a `value` of type `null | undefined`:

```
function isNumberOrString(value: unknown): value is number | string {
    return ['number', 'string'].includes(typeof value);
}

function logValueIfExists(value: number | string | null | undefined) {
    if (isNumberOrString(value)) {
        // Type of value: number | string
        value.toString(); // Ok
    } else {
        // Type of value: null | undefined
        console.log("value does not exist:", value);
    }
}
```

You can think of a type predicate as returning not just a boolean, but also an indication that the argument was that more specific type.

Type predicates are often used to check whether an object already known to be an instance of one interface is an instance of a more specific interface.

Here, the `StandupComedian` interface contains additional information on top of `Comedian`. The `isStandupComedian` type guard can be used to check whether a general `Comedian` is specifically a `StandupComedian`:

```
interface Comedian {
    funny: boolean;
```

```

}

interface StandupComedian extends Comedian {
    routine: string;
}

function isStandupComedian(value: Comedian): value is StandupComedian {
    return 'routine' in value;
}

function workWithComedian(value: Comedian) {
    if (isStandupComedian(value)) {
        // Type of value: StandupComedian
        console.log(value.routine); // Ok
    }

    // Type of value: Comedian
    console.log(value.routine);
    //
    // Error: Property 'routine' does not exist on type 'Comedian'.
}

```

Be warned: because type predicates also narrow types in the false case, you might get surprising results if a type predicate checks more than just the type of its input.

This `isLongString` type predicate returns `false` if its `input` parameter is `undefined` or a `string` with a length less than 7. As a result, the `else` statement (its false case) is narrowed to thinking `text` must be type `undefined`:

```

function isLongString(input: string | undefined): input is string {
    return !(input && input.length >= 7);
}

function workWithText(text: string | undefined) {
    if (isLongString(text)) {
        // Type of text: string
        console.log("Long text:", text.length);
    } else {
        // Type of text: undefined
        console.log("Short text:", text?.length);
        //
        // Error: Property 'length' does not exist on type 'never'.
    }
}

```

```
}  
}
```

Type predicates that do more than verify the type of a property or value are easy to misuse. I generally recommend avoiding them when possible. Simpler type predicates are sufficient for most cases.

Type Operators

Not all types can be represented using only a keyword or a name of an existing type. It can sometimes be necessary to create a new type that combines both, performing some transformation on the properties of an existing type.

keyof

JavaScript objects can have members retrieved using dynamic values, which are commonly (but not necessarily) `string` typed. Representing these keys in the type system can be tricky. Using a catchall primitive such as `string` would allow invalid keys for the container value.

That's why TypeScript when using stricter configuration settings—covered in Part 13 - Configuration Options,—would report an error on the `ratings[key]` as seen in the next example. Type `string` allows values not allowed as properties on the `Ratings` interface, and `Ratings` doesn't declare an index signature to allow any `string` keys:

```
interface Ratings {  
    audience: number;  
    critic: number;  
}  
  
function getRating(ratings: Ratings, key: string): number {  
    return ratings[key];  
    //      ~~~~~  
    // Error: Element implicitly has an 'any' type because expression  
    // of type 'string' can't be used to index type 'Ratings'.  
    //   No index signature with a parameter of  
    //   type 'string' was found on type 'Ratings'.  
}  
  
const ratings: Ratings = { audience: 66, critic: 84 };
```

```
getRating(ratings, 'audience'); // Ok
```

```
getRating(ratings, 'not valid'); // Ok, but shouldn't be
```

Another option would be to use a type union of literals for the allowed keys. That would be more accurate in properly restricting to only the keys that exist on the container value:

```
function getRating(ratings: Ratings, key: 'audience' | 'critic'): number {  
    return ratings[key]; // Ok  
}
```

```
const ratings: Ratings = { audience: 66, critic: 84 };
```

```
getRating(ratings, 'audience'); // Ok
```

```
getRating(ratings, 'not valid');
```

```
//
```

```
// Error: Argument of type '"not valid"' is not
```

```
// assignable to parameter of type '"audience" | "critic"'.  
~~~~~
```

However, what if the interface has dozens or more members? You would have to type out each of those members' keys into the union type and keep them up-to-date. What a pain.

TypeScript instead provides a `keyof` operator that takes in an existing type and gives back a union of all the keys allowed on that type. Place it in front of the name of a type wherever you might use a type, such as a type annotation.

Here, `keyof Ratings` is equivalent to `'audience' | 'critic'` but is much quicker to write out and won't need to be manually updated if the `Ratings` interface ever changes:

```
function getCountKeyof(ratings: Ratings, key: keyof Ratings): number {  
    return ratings[key]; // Ok  
}
```

```
const ratings: Ratings = { audience: 66, critic: 84 };
```

```
getCountKeyof(ratings, 'audience'); // Ok
```

```

getCountKeyof(ratings, 'not valid');
//
// Error: Argument of type '"not valid"' is not
// assignable to parameter of type 'keyof Ratings'.

```

`keyof` is a great feature for creating union types based on the keys of existing types. It also combines well with other type operators in TypeScript, allowing for some very nifty patterns you'll see later in this part and Part 15 - Type Operations .

typeof

Another type operator provided by TypeScript is `typeof` . It gives back the type of a provided value. This can be useful if the value's type would be annoyingly complex to write manually.

Here, the `adaptation` variable is declared as being the same type as `original` :

```

const original = {
  medium: "movie",
  title: "Mean Girls",
};

let adaptation: typeof original;

if (Math.random() > 0.5) {
  adaptation = { ...original, medium: "play" }; // Ok
} else {
  adaptation = { ...original, medium: 2 };
  //
  // Error: Type 'number' is not assignable to type 'string'.
}

```

Although the `typeof` type operator visually looks like the *runtime* `typeof` operator used to return a string description of a value's type, the two are different. They only coincidentally use the same word. Remember: the JavaScript operator is a runtime operator that returns the string name of a type. The TypeScript version, because it's a type operator, can only be used in types and won't appear in compiled code.

keyof typeof

`typeof` retrieves the type of a value, and `keyof` retrieves the allowed keys on a type. TypeScript allows the two keywords to be chained together to succinctly retrieve the allowed keys on a value's type. Putting them together, the `typeof` type operator becomes wonderfully useful for working with `keyof` type operations.

In this example, the `logRating` function is meant to take in one of the keys of the `ratings` value. Instead of creating an interface, the code uses `keyof typeof` to indicate `key` must be one of the keys on the type of the `ratings` value:

```
const ratings = {
  imdb: 8.4,
  metacritic: 82,
};

function logRating(key: keyof typeof ratings) {
  console.log(ratings[key]);
}

logRating("imdb"); // Ok

logRating("invalid");
//           ~~~~~
// Error: Argument of type '"invalid"' is not assignable
// to parameter of type '"imdb" | "metacritic"'.
```

By combining `keyof` and `typeof`, we get to save ourselves the pain of writing out—and having to update—types representing the allowed keys on objects that don't have an explicit interface type.

Type Assertions

TypeScript works best when your code is “strongly typed”: all the values in your code have precisely known types. Features such as top types and type guards provide ways to wrangle complex code into being understood by TypeScript's type checker. However, sometimes it's not reasonably possible to be 100% accurate in telling the type system how your code is meant to work.

For example, `JSON.parse` intentionally returns the top type `any`. There's no way to safely inform the type system that a particular string value given to `JSON.parse`

should return any particular value type. (As we will see in Part 10 - Generics , adding a generic type to `parse` that is only used once for a return type would violate a best practice known as The Golden Rule of Generics.)

TypeScript provides a syntax for overriding the type system's understanding of a value's type: a "type assertion," also known as a "type cast." On a value that is meant to be a different type, you can place the `as` keyword followed by a type. TypeScript will defer to your assertion and treat the value as that type.

In this snippet, it is possible that the returned result from `JSON.parse` is meant to be a type such as `string[]`, `[string, string]`, or `["grace", "frankie"]`. The snippet uses type assertions for three of the lines of code to switch the type from `any` to one of those:

```
const rawData = `["grace", "frankie"]`;

// Type: any
JSON.parse(rawData);

// Type: string[]
JSON.parse(rawData) as string[];

// Type: [string, string]
JSON.parse(rawData) as [string, string];

// Type: ["grace", "frankie"]
JSON.parse(rawData) as ["grace", "frankie"];
```

Type assertions exist only in the TypeScript type system. They're removed along with all other pieces of type system syntax when compiled to JavaScript. The previous code would look like this when compiled to JavaScript:

```
const rawData = `["grace", "frankie"]`;

// Type: any
JSON.parse(rawData);

// Type: string[]
JSON.parse(rawData);

// Type: [string, string]
JSON.parse(rawData);
```

```
// Type: ["grace", "frankie"]
JSON.parse(rawData);
```

NOTE

If you're working with older libraries or code, you may see a different casting syntax that looks like `<type>item` instead of `item as type`. Because this syntax is incompatible with JSX syntax and therefore does not work in `.tsx` files, it is discouraged.

TypeScript best practice is generally to avoid using type assertions when possible. It's best for your code to be fully typed and to not need to interfere with TypeScript's understanding of its types using assertions. But occasionally there will be cases where type assertions are useful, even necessary.

Asserting Caught Error Types

Error handling is another place where type assertions may come in handy. It is generally impossible to know what type a caught error in a `catch` block will be because the code in the `try` block may unexpectedly throw any object different from what you expect. Furthermore, although JavaScript best practice is to always throw an instance of the `Error` class, some projects instead throw string literals or other surprising values.

If you are absolutely confident that an area of code will only throw an instance of the `Error` class, you can use a type assertion to treat a caught assertion as an `Error`. This snippet accesses the `message` property of a caught `error` that it assumes is an instance of the `Error` class:

```
try {
  // (code that may throw an error)
} catch (error) {
  console.warn("Oh no!", (error as Error).message);
}
```

It is generally safer to use a form of type narrowing such as an `instanceof` check to ensure the thrown error is the expected error type. This snippet checks whether the thrown error is an instance of the `Error` class to know whether to log that message or the error itself:

```
try {
    // (code that may throw an error)
} catch (error) {
    console.warn("Oh no!", error instanceof Error ? error.message : error);
}
```

Non-Null Assertions

Another common use case for type assertions is to remove `null` and/or `undefined` from a variable that only theoretically, not practically, might include them. That situation is so common that TypeScript includes a shorthand for it. Instead of writing out `as` and the full type of whatever a value is excluding `null` and `undefined`, you can use a `!` to signify the same thing. In other words, the `!` non-null assertion asserts that the type is not `null` or `undefined`.

The following two type assertions are identical in that they both result in `Date` and not `Date | undefined`:

```
// Inferred type: Date | undefined
let maybeDate = Math.random() > 0.5
    ? undefined
    : new Date();

// Asserted type: Date
maybeDate as Date;

// Asserted type: Date
maybeDate!;
```

Non-null assertions are particularly useful with APIs such as `Map.get` that return a value or `undefined` if it doesn't exist.

Here, `seasonCounts` is a general `Map<string, number>`. We know that it contains an `"I Love Lucy"` key so the `knownValue` variable can use a `!` to remove `| undefined` from its type:

```
const seasonCounts = new Map([
    ["I Love Lucy", 6],
    ["The Golden Girls", 7],
]);
```

```
// Type: number | undefined
const maybeValue = seasonCounts.get("I Love Lucy");

console.log(maybeValue.toPrecision());
//           ~~~~~
// Error: Object is possibly 'undefined'.

// Type: number
const knownValue = seasonCounts.get("I Love Lucy")!;

console.log(knownValue.toPrecision()); // Ok
```

Type Assertion Caveats

Type assertions, like the `any` type, are a necessary escape hatch for TypeScript's type system. Therefore, also like the `any` type, they should be avoided whenever reasonably possible. It is often better to have more accurate types representing your code than it is to make it easier to assert on a value's type. Those assertions are often wrong—either already so at the time of writing, or they become wrong later on as the codebase changes.

For example, suppose the `seasonCounts` example were to change over time to have different values in the map. Its non-null assertion might still make the code pass TypeScript type checking, but there might be a runtime error:

```
const seasonCounts = new Map([
  ["Broad City", 5],
  ["Community", 6],
]);

// Type: string
const knownValue = seasonCounts.get("I Love Lucy")!;

console.log(knownValue.toPrecision()); // No type error, but...
// Runtime TypeError: Cannot read property 'toPrecision' of undefined.
```

Type assertions should generally be used sparingly, and only when you're absolutely certain it is safe to do so.

Assertions versus declarations

There is a difference between using a type annotation to declare a variable's type versus using a type assertion to change the type of a variable with an initial value. TypeScript's type checker performs assignability checking on a variable's initial value against the variable's type annotation when both exist. A type assertion, however, explicitly tells TypeScript to skip some of its type checking.

The following code creates two objects of type `Entertainer` with the same flaw: a missing `acts` member. TypeScript is able to catch the error in the `declared` variable because of its `: Entertainer` type annotation. It is not able to catch the error on the `asserted` variable because of the type assertion:

```
interface Entertainer {
  acts: string[];
  name: string;
}

const declared: Entertainer = {
  name: "Moms Mabley",
};
// Error: Property 'acts' is missing in type
// '{ name: string; }' but required in type 'Entertainer'.

const asserted = {
  name: "Moms Mabley",
} as Entertainer; // Ok, but...

// Both of these statements would fail at runtime with:
// Runtime TypeError: Cannot read properties of undefined (reading 'join')
console.log(declared.acts.join(", "));
console.log(asserted.acts.join(", "));
```

It is therefore strongly preferable to either use a type annotation or allow TypeScript to infer a variable's type from its initial value.

Assertion assignability

Type assertions are meant to be only a small escape hatch, for situations where some value's type is slightly incorrect. TypeScript will only allow type assertions between two types if one of the types is assignable to the other. If the type assertion is

between two completely unrelated types, then TypeScript will notice and report a type error.

For example, switching from one primitive to another is not allowed, as primitives have nothing to do with each other:

```
let myValue = "Stella!" as number;
//
// Error: Conversion of type 'string' to type 'number'
// may be a mistake because neither type sufficiently
// overlaps with the other. If this was intentional,
// convert the expression to 'unknown' first.
```

If you absolutely must switch a value from one type to a totally unrelated type, you can use a double type assertion. First cast the value to a top type—`any` or `unknown`—and then cast that result to the unrelated type:

```
let myValueDouble = "1337" as unknown as number; // Ok, but... eww.
```

`as unknown as...` double type assertions are dangerous and almost always a sign of something incorrect in the types of the surrounding code. Using them as an escape hatch from the type system means the type system may not be able to save you when changes to surrounding code would cause an issue with previously working code. I teach double type assertions only as a precautionary tale to help explain the type system, not to encourage their use.

Const Assertions

Back in Part 6 Arrays, I introduced an `as const` syntax for changing a mutable array type to a read-only tuple type and promised to use it more later in the material. That time is now!

Const assertions can generally be used to indicate that any value—array, primitive, value, you name it—should be treated as the constant, immutable version of itself. Specifically, `as const` applies the following three rules to whatever type it receives:

- Arrays are treated as `readonly` tuples, not mutable arrays.
- Literals are treated as literals, not their general primitive equivalents.

- Properties on objects are considered `readonly`.

You've already seen arrays become tuples, as with this array being asserted as a tuple:

```
// Type: (number | string)[]  
[0, ''];  
  
// Type: readonly [0, '']  
[0, ''] as const;
```

Let's dig into the other two changes `as const` produces.

Literals to Primitives

It can be useful for the type system to understand a literal value to be that specific literal, rather than widening it to its general primitive.

For example, similar to functions that return tuples, it might be useful for a function to be known to produce a specific literal instead of a general primitive. These functions also return values that can be made more specific—here, `getNameConst`'s return type is the more specific `"Maria Bamford"` instead of the general `string`:

```
// Type: () => string  
const getName = () => "Maria Bamford";  
  
// Type: () => "Maria Bamford"  
const getNameConst = () => "Maria Bamford" as const;
```

It may also be useful to have specific fields on a value be more specific literals. Many popular libraries ask that a discriminant field on a value be a specific literal so the types of their code can more specifically make inferences on the value. Here, the `narrowJoke` variable has a `style` of type `"one-liner"` instead of `string`, so it can be provided in a location that needs type `Joke`:

```
interface Joke {  
  quote: string;  
  style: "story" | "one-liner";  
}
```



```
function tellJoke(joke: Joke) {
    if (joke.style === "one-liner") {
        console.log(joke.quote);
    } else {
        console.log(joke.quote.split("\n"));
    }
}

// Type: { quote: string; style: "one-liner" }
const narrowJoke = {
    quote: "If you stay alive for no other reason do it for spite.",
    style: "one-liner" as const,
};

tellJoke(narrowJoke); // Ok

// Type: { quote: string; style: string }
const wideObject = {
    quote: "Time flies when you are anxious!",
    style: "one-liner",
};

tellJoke(wideObject);
// Error: Argument of type '{ quote: string; style: string; }'
// is not assignable to parameter of type 'Joke'.
//   Types of property 'style' are incompatible.
//     Type 'string' is not assignable to type '"story" | "one-liner"'.
```

Read-Only Objects

Object literals such as those used as the initial value of a variable generally widen the types of properties the same way the initial values of `let` variables widen. String values such as `'apple'` become primitives such as `string`, arrays are typed as arrays instead of tuples, and so on. This can be inconvenient when some or all of those values are meant to later be used in a place that requires their specific literal type.

Asserting a value literal with `as const`, however, switches the inferred type to be as specific as possible. All member properties become `readonly`, literals are considered their own literal type instead of their general primitive type, arrays become read-only tuples, and so on. In other words, applying a `const` assertion to a value literal makes that value literal immutable and recursively applies the same `const` assertion logic to all its members.

As an example, the `preferencesMutable` value that follows is declared without an `as const`, so its names are the primitive type `string` and it's allowed to be modified. `preferencesReadonly`, however, is declared with an `as const`, so its member values are literals and not allowed to be modified:

```
function describePreference(preference: "maybe" | "no" | "yes") {
  switch (preference) {
    case "maybe":
      return "I suppose...";
    case "no":
      return "No thanks.";
    case "yes":
      return "Yes please!";
  }
}

// Type: { movie: string, standup: string }
const preferencesMutable = {
  movie: "maybe",
  standup: "yes",
};

describePreference(preferencesMutable.movie);
// ~~~~~
// Error: Argument of type 'string' is not assignable
// to parameter of type '"maybe" | "no" | "yes"'.

preferencesMutable.movie = "no"; // Ok

// Type: readonly { readonly movie: "maybe", readonly standup: "yes" }
const preferencesReadonly = {
  movie: "maybe",
  standup: "yes",
} as const;

describePreference(preferencesReadonly.movie); // Ok

preferencesReadonly.movie = "no";
// ~~~~~
// Error: Cannot assign to 'movie' because it is a read-only property.
```

In this part, you used type modifiers to take existing objects and/or types and turn them into new types:

- Top types: the highly permissive `any` and the highly restrictive `unknown`
- Type operators: using `keyof` to grab the keys of a type and/or `typeof` to grab the type of a value
- Using—and when not to use—type assertions to sneakily change the type of a value
- Narrowing types using `as const` assertions